

Suite ODIP : Création d’environnements de test

Jérémy Billuart

11 juillet 2025

Introduction

Dans le cadre d’un déploiement opérationnel potentiel d’ODIP, nous avons entrepris d’améliorer les performances de notre système. Deux axes majeurs sont au cœur de cette optimisation : la précision de la classification et la vitesse de traitement.

En attendant un accès à la base de données CALIOPE, qui permettrait de réentraîner un classifieur plus performant et plus généraliste, nous avons choisi de nous concentrer sur la réduction du temps de traitement. Étant donné que le classifieur actuel d’ODIP n’est probablement pas définitif, nous n’avons pas appliqué d’optimisations directes du modèle.

À la place, nous avons introduit une logique d’ordonnancement intelligent des images à classifier, visant à explorer en priorité les dossiers et sous-dossiers les plus susceptibles de contenir des images à caractère pédopornographique. L’objectif est de fouiller et de trouver le plus rapidement possible les dossiers contenant des contenus illicites.

Ce projet relève du *Reinforcement Learning* (RL) et, comme souvent dans ce domaine, nécessite un compromis entre exploration et exploitation. Les métriques que nous cherchons à optimiser sont définies en ANNEXE A.

Ce document présente les démarches et techniques mises en œuvre pour générer des environnements de test variés, permettant l’évaluation de l’agent RL en cours de développement. Les hyperparamètres utilisés seront indiqués en *italique*, et détaillés dans la section 3.

Dans la suite de ce document, nous décrirons souvent une arborescence de dossiers comme un arbre enraciné, faiblement connexe, avec une racine, des nœuds internes et des feuilles.

1 Description du système

L’objectif est de mesurer les performances de l’agent dans un large éventail de situations. N’ayant ni connaissance précise ni structure typique de l’organisation de l’espace de stockage d’un pédocriminel, nous générons automatiquement un grand nombre d’environnements très variés pour évaluer l’agent sur des situations opposées et contrastées.

Le système de génération d’environnements de test comprend deux composantes principales :

1. **Génération de l’arborescence** : création de la structure hiérarchique des dossiers.
2. **Distribution des fichiers** : pour chaque arborescence, création de copies avec des répartitions différentes de fichiers positifs et négatifs.

Que ce soit dans la génération d’arborescence ou la distribution des fichiers, les paramètres imposent des contraintes souples et flexibles. Si l’algorithme ne permet pas de générer un environnement avec ces hyperparamètres, il s’adapte, ce qui ne pose aucun problème car l’objectif est uniquement de produire des environnements variés. En grande majorité, les contraintes sont respectées, et sinon, l’environnement créé reste très proche de celui espéré.

1.1 Génération de l'arborescence

Pour décrire l'approche utilisée pour la génération de l'arborescence, nous allons d'abord décrire de manière théorique, puis nous illustrons ces étapes avec l'aide d'un modèle-jouet.

Bottom-Up

La génération d'arborescences repose sur une approche bottom-up qui simule un processus de construction naturel partant des feuilles vers la racine. Cette stratégie présente l'avantage de produire des structures arborescentes plus réalistes qu'une approche top-down, en évitant notamment la création d'arbres trop réguliers.

L'algorithme débute par la création de $n_{initial}$ nœuds isolés (cf ANNEXE B pour le calcul de cette borne), chacun représentant une feuille potentielle de l'arborescence finale. Ces nœuds constituent une forêt initiale où chaque élément est considéré comme un arbre de profondeur zéro.

Processus de fusion itératif

Le cœur de l'algorithme consiste en un processus de fusion itératif qui agrège progressivement les arbres de la forêt. À chaque itération, l'algorithme sélectionne aléatoirement entre 2 et $max_{children}$ arbres candidats parmi ceux qui respectent encore la contrainte de max_{height} . Cette sélection privilégie les arbres dont la fusion n'entraînerait pas un dépassement de la hauteur limite fixée.

Une fois les candidats sélectionnés, un nouveau nœud parent est créé et connecté aux racines des arbres choisis. La profondeur de ce nouveau sous-arbre est calculée comme étant égale à un plus la profondeur maximale des arbres fusionnés. Ce mécanisme garantit une propagation correcte des informations de profondeur tout au long du processus de construction.

Gestion des contraintes et ajustements

L'algorithme intègre plusieurs mécanismes de régulation pour respecter les contraintes imposées. À la fin du processus de fusion des arbres, une phase d'ajustement intervient pour supprimer les nœuds excédentaires. Cette suppression s'effectue de manière contrôlée en identifiant les nœuds supprimables : les feuilles qui peuvent être éliminées sans affecter la structure, et les nœuds intermédiaires à un seul enfant qui peuvent être contournés en reconnectant directement leur parent à leur enfant.

La stratégie de suppression préserve la connectivité de l'arborescence. Cette approche permet d'obtenir des arborescences dont le nombre de nœuds correspond à n_{nodes} .

Modèle-jouet

Pour illustrer les différentes étapes de génération d'une arborescence, nous utilisons un modèle-jouet avec les paramètres suivants : $n_{nodes}=15$, $max_{children}=3$, et $max_{height}=4$.

La génération débute par la création de $n_{initial}=11$ nœuds isolés, comme illustré à la Figure 1.

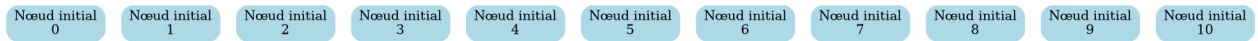


FIGURE 1 – Initialisation du graphe

Un processus de fusion itératif est ensuite lancé afin de construire progressivement l'arborescence en agrégeant les différents sous-graphes.

La Figure 2 présente l'état du graphe après 4 opérations de fusion :

- La première fusion crée le nœud 11 en reliant les nœuds 8 et 9.
- La deuxième fusion génère le nœud 12 en connectant les nœuds 5 et 7.
- La troisième fusion crée le nœud 13 à partir des nœuds 0 et 11.
- Enfin, la quatrième fusion produit le nœud 14 en reliant les nœuds 2, 4 et 10.

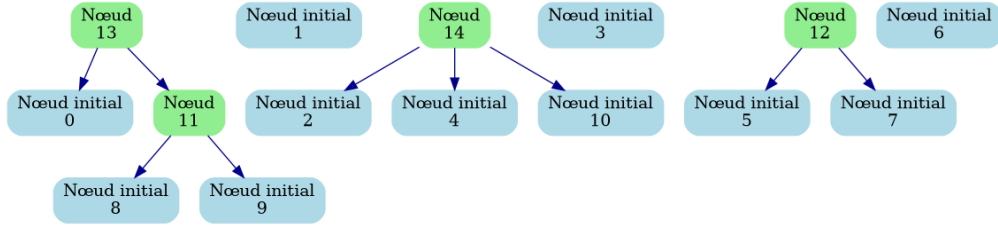


FIGURE 2 – État du graphe après 4 fusions

Une fois toutes les fusions réalisées, un unique arbre est obtenu. Le dernier nœud créé (le 17) devient la racine de l'arborescence, comme illustré à la Figure 3.

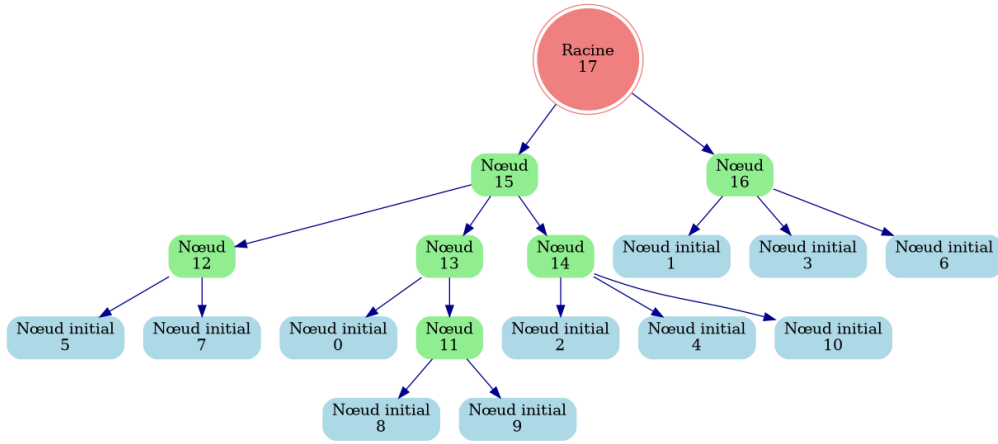


FIGURE 3 – Arbre complet obtenu après la dernière fusion

Cependant, le processus de fusions a engendré des nœuds excédentaires. Pour respecter la contrainte n_{nodes} , l'arbre est ensuite élagué. Dans l'exemple ci-dessous (Figure 4), les nœuds 10, 1 et 3 ont été supprimés.

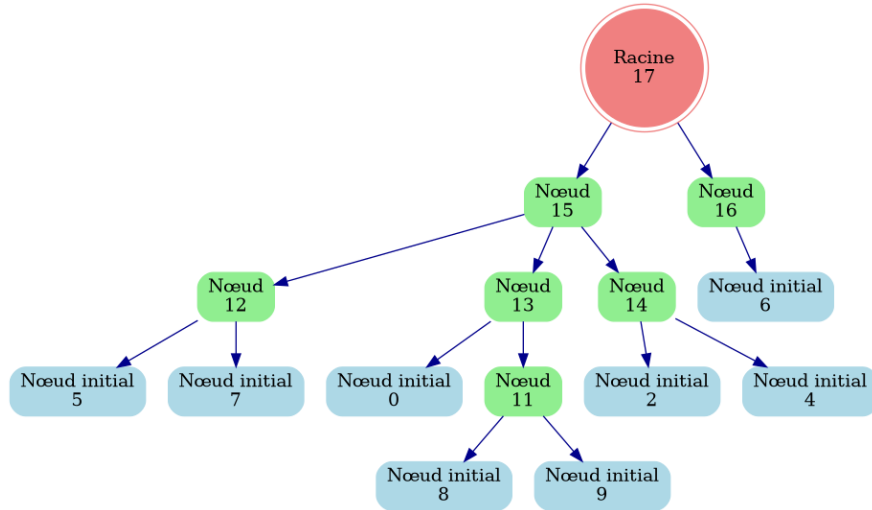


FIGURE 4 – Arborescence finale après élagage

Nous obtenons bien un graphe respectant les contraintes : n_{nodes} , $max_{children}$, et max_{height} .

1.2 Stratégie de distribution des fichiers

1.2.1 Sélection des candidats

Cette étape du processus de distribution vise à identifier, parmi les nœuds du graphe, ceux qui sont les plus pertinents pour devenir les centroïdes des clusters positifs. Nous avons conçu notre algorithme de sélection de manière à ce que les candidats proposés couvrent des zones variées du graphe, afin de maximiser la diversité spatiale.

Plusieurs stratégies de sélection ont été explorées. Pour les comparer, nous présentons d'abord le principe de fonctionnement de chaque algorithme, puis illustrons, à partir d'un même graphe en forme d'arborescence, les 10 candidats qu'ils sélectionnent respectivement.

Farthest Point Sampling avec initialisation aléatoire

Cette méthode suit trois étapes successives :

1. Sélection d'un nœud aléatoire (différent de la racine).
2. Sélection du nœud le plus éloigné de ce premier point.
3. Itérativement, ajout du nœud qui maximise la distance minimale par rapport à l'ensemble des nœuds déjà sélectionnés.

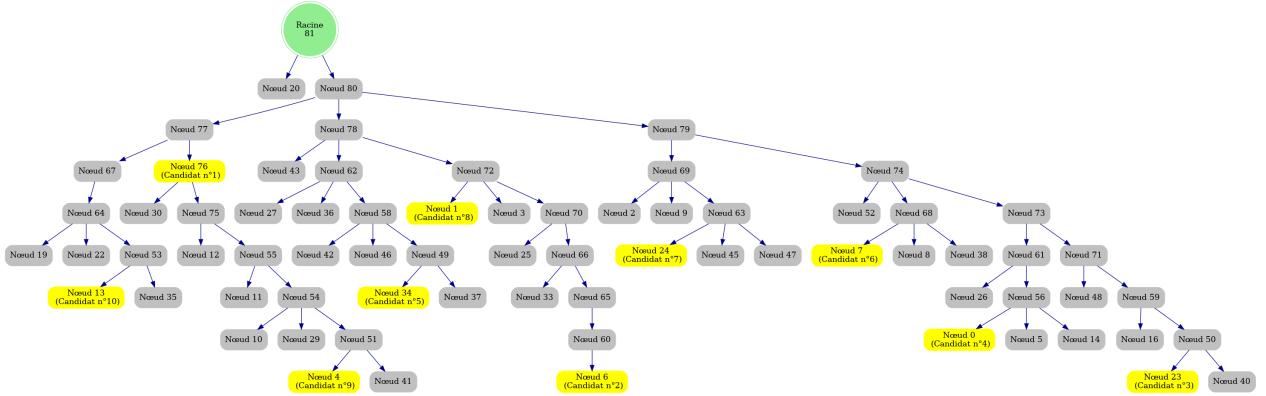


FIGURE 5 – FPS avec initialisation aléatoire (FPSRI)

Farthest Point Sampling avec initialisation déterministe

Cette variante est identique à la précédente, à une différence près : le point de départ n'est pas choisi au hasard, mais correspond à l'un des nœuds les plus profonds du graphe, c'est-à-dire le plus éloigné de la racine.

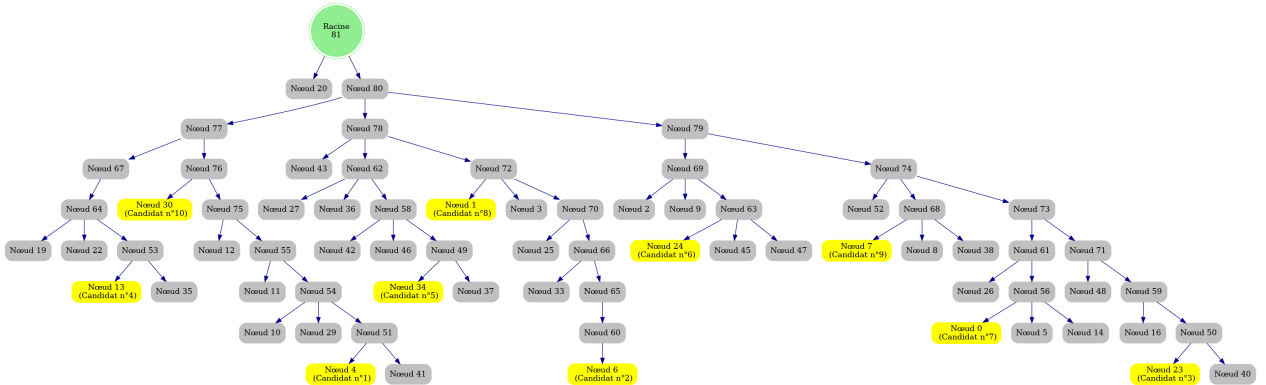


FIGURE 6 – FPS avec initialisation déterministe (FPSDI)

Sélection aléatoire

Cette approche consiste simplement à sélectionner des nœuds aléatoires en excluant la racine.

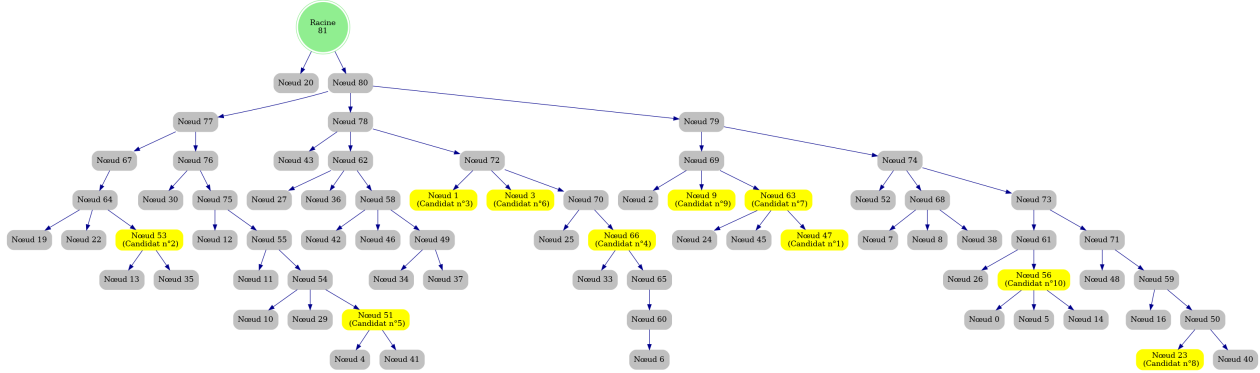


FIGURE 7 – Sélection aléatoire

Choix de la méthode de sélection

Les deux variantes de l'algorithme FPS (FPSRI et FPSDI) produisent des distributions de candidats très similaires, montrant que l'impact de l'initialisation est négligeable. Dans les deux cas, les candidats sont majoritairement localisés vers les feuilles du graphe. Cela s'explique naturellement par la structure d'un graphe enraciné, où les nœuds les plus éloignés se trouvent aux extrémités.

Ce comportement induit un biais dans la localisation des clusters positifs, mais il est cohérent avec notre hypothèse : dans une logique de dissimulation de contenu pédopornographique, il est plus plausible que ce contenu soit caché en profondeur dans l'arborescence, plutôt qu'à proximité de la racine.

La stratégie de sélection aléatoire, illustrée en Figure 7, montre une répartition plus irrégulière des candidats dans l'arborescence. Certains sont proches de la racine, tandis que d'autres se trouvent en profondeur. Cette dispersion peu contrôlée peut entraîner une couverture spatiale moins homogène et un manque de pertinence dans le choix des centroïdes. Ce caractère aléatoire rend donc cette méthode moins adaptée à notre objectif, qui est d'obtenir une bonne couverture de l'arborescence tout en minimisant la redondance entre candidats.

Nous retenons donc l'algorithme FPS avec initialisation aléatoire pour la suite de nos travaux, car il offre une distribution spatiale optimale des candidats tout en étant moins coûteux en calcul que la version déterministe.

Organisation en clusters et sélection des emplacements

La distribution des fichiers positifs repose sur la création de clusters positifs, chacun regroupant un certain nombre de dossiers positifs. Le paramètre $n_clusters$ fixe le nombre total de clusters à générer, tandis que n_pos_folder détermine le nombre total de dossiers positifs à distribuer dans l'arborescence. La répartition est effectuée de manière à ce que chaque cluster positif contienne au moins un dossier positif, et que la somme sur l'ensemble des clusters soit égale à n_pos_folder .

Pour identifier les emplacements candidats, le système s'appuie sur l'algorithme de sélection décrit précédemment (FPSRI). Pour un cluster positif donné, le système considère successivement tous les candidats encore disponibles (c'est-à-dire non utilisés pour un autre cluster). Autour de chaque candidat, il extrait un sous-graphe défini par un rayon max_gap . Si ce sous-graphe ne contient pas suffisamment de nœuds pour accueillir le nombre de dossiers positifs attribués au cluster, max_gap est progressivement augmenté jusqu'à atteindre la taille requise.

Ainsi, chaque cluster positif est associé à un ensemble de sous-graphes potentiels, chacun centré sur un candidat différent. Ces sous-graphes constituent les zones de placement possibles pour les dossiers positifs.

Dans la Figure 8, les nœuds appartenant à un même sous-graphe sont représentés par la même couleur. On observe que les sous-graphes centrés sur différents candidats couvrent une large portion du graphe, ce qui garantit la diversité des environnements générés.

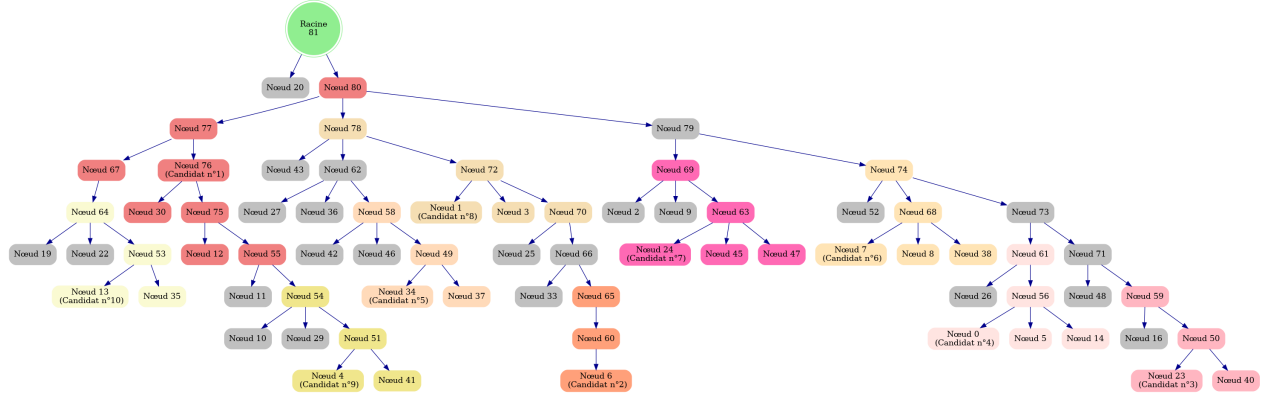


FIGURE 8 – Représentation des clusters positifs à partir de sous-graphes centrés sur des candidats FPSRI

Optimisation par combinaisons

Pour chaque cluster positif et pour chaque sous-graphe associé à un candidat disponible, le système génère un ensemble de combinaisons de nœuds (un maximum par sous-graphe afin de maîtriser la complexité). Chaque combinaison contient exactement le nombre de dossiers positifs attribués au cluster considéré.

Chaque combinaison est évaluée selon un score basé sur l'écart entre la distance moyenne réelle des nœuds sélectionnés et la *mean_exp_dist* cible. La combinaison minimisant cet écart est retenue comme solution optimale pour le cluster. Les nœuds appartenant à cette combinaison sont alors désignés comme nœuds positifs. La distance moyenne utilisée pour l'évaluation correspond à la moyenne des distances minimales entre chaque nœud et ses voisins les plus proches dans la sélection. Cette métrique capture la compacité du cluster tout en évitant les configurations où certains nœuds seraient isolés des autres.

1.2.2 Distribution des fichiers

Distribution des fichiers positifs

La distribution des fichiers positifs se fait également de manière aléatoire, tout en garantissant que chaque dossier positif contient au moins un fichier positif et que la somme des fichiers positifs soit égale à n_pos_img .

Distribution des fichiers négatifs

Les fichiers négatifs sont répartis aléatoirement dans les nœuds. À terme, un hyperparamètre supplémentaire pourrait être introduit pour contrôler le nombre de fichiers négatifs dans les dossiers positifs, ajoutant ainsi une forme de bruit réaliste. Ce bruit est déjà introduit dans notre système, car le classifieur utilisé produit des faux positifs et des faux négatifs. De plus un autre bruit existe naturellement car tous les dossiers voisins d'un dossier positifs ne sont pas forcément un dossier positifs.

2 Résultats

Pour illustrer les environnements de test créés, nous présentons deux arborescences générées à partir d'hyperparamètres différents. Ensuite, nous définissons deux distributions, elles-mêmes déterminées par leurs hyperparamètres, que nous appliquons successivement sur chacune des deux arborescences.

Arborescences générées

Arborescence n°1 Hyperparamètres utilisés :

- *n_nodes* = 93
- *max_children* = 4
- *max_height* = 13

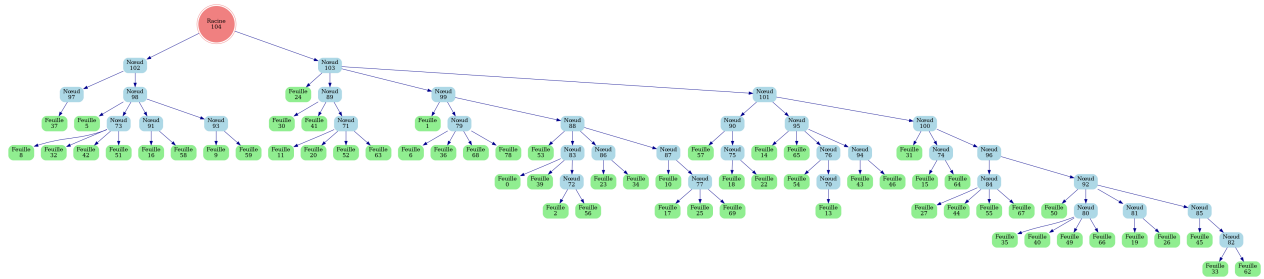


FIGURE 9 – Arborescence n°1

Arborescence n°2 Hyperparamètres utilisés :

- *n_nodes* = 136
- *max_children* = 4
- *max_height* = 12

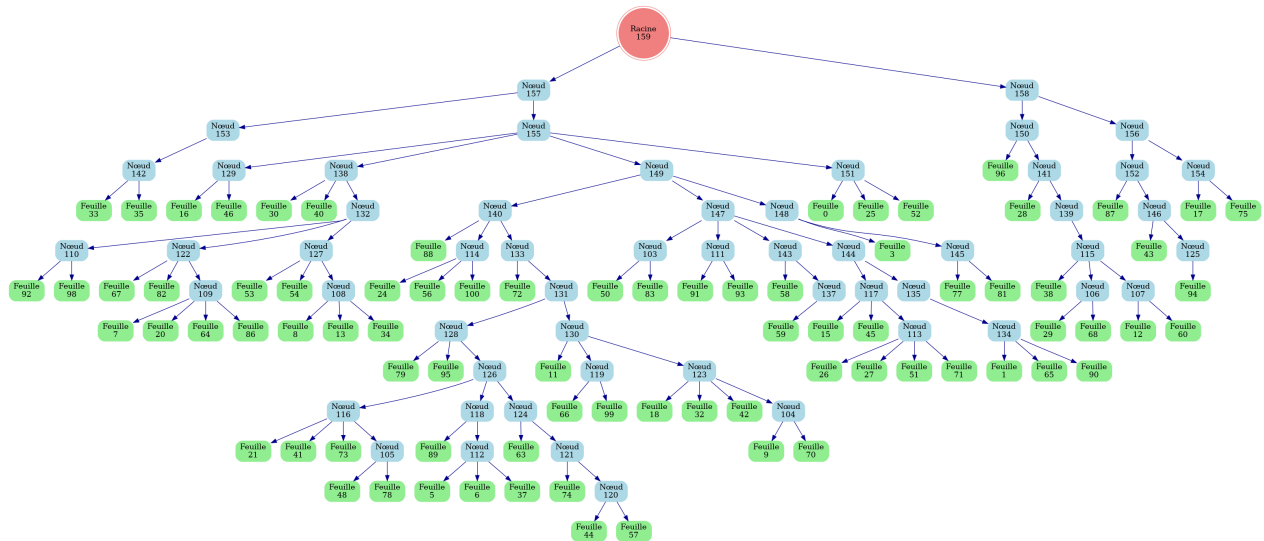


FIGURE 10 – Arborescence n°2

Distributions appliquées

Distribution n°1 Hyperparamètres utilisés :

- $n_clusters = 1$
- $n_pos_folder = 5$
- $n_pos_img = 90$
- $n_neg_img = 744$
- $max_gap = 3$
- $mean_exp_dist = 1.53$

Distribution n°2 Hyperparamètres utilisés :

- $n_clusters = 3$
- $n_pos_folder = 14$
- $n_pos_img = 164$
- $n_neg_img = 335$
- $max_gap = 2$
- $mean_exp_dist = 1.59$

Résultats obtenus

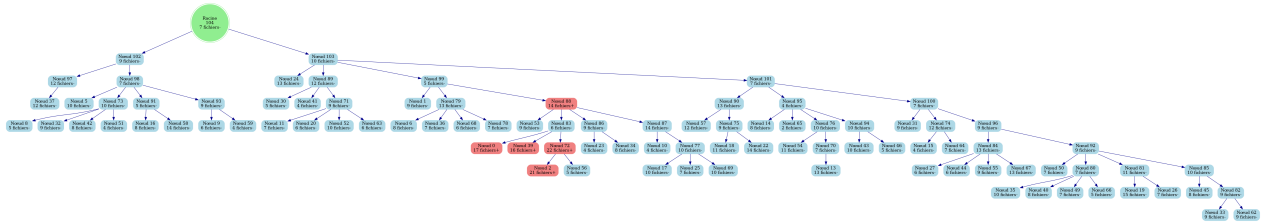


FIGURE 11 – Arborescence n°1 avec la distribution n°1

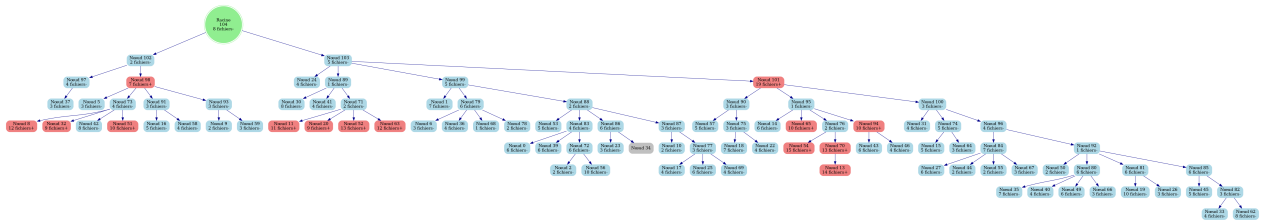


FIGURE 12 – Arborescence n°1 avec la distribution n°2

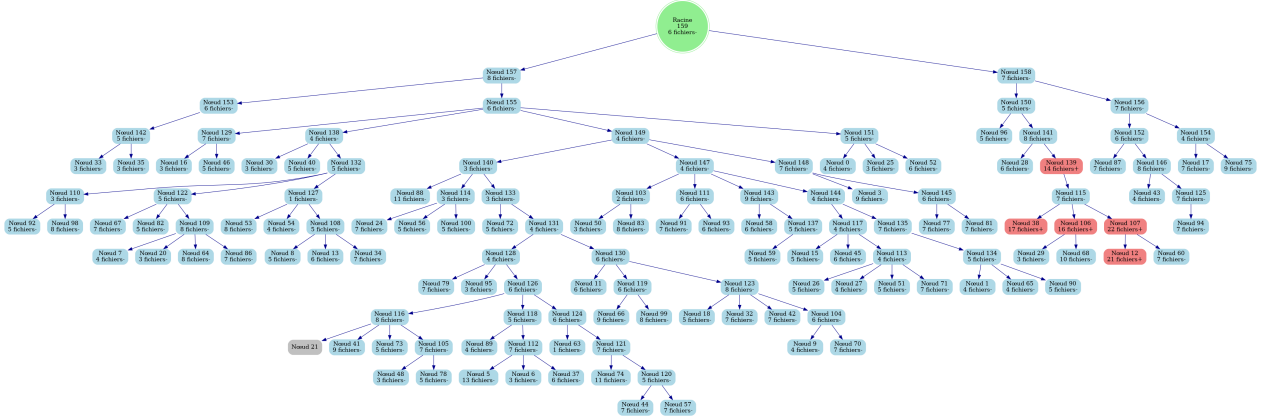


FIGURE 13 – Arborescence n°2 avec la distribution n°1

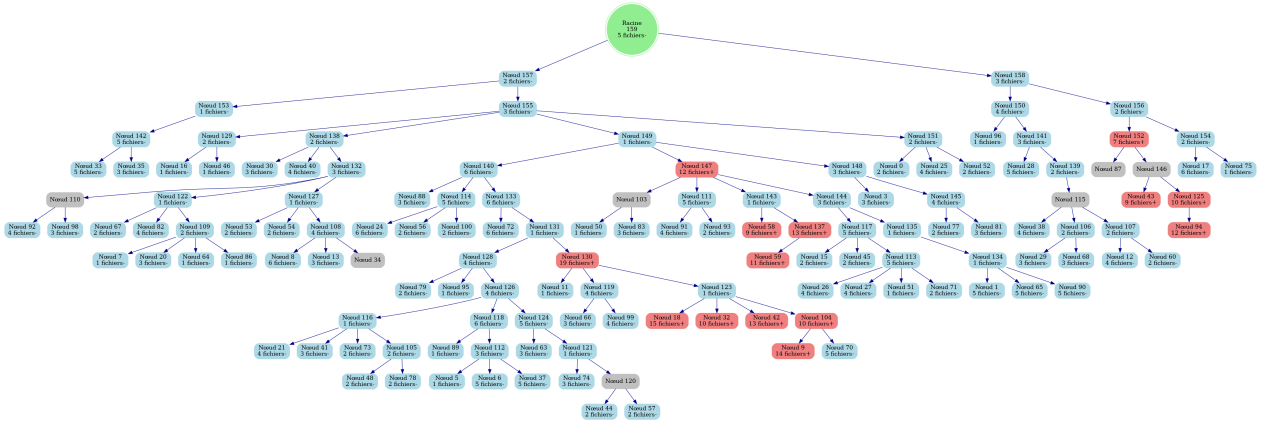


FIGURE 14 – Arborescence n°2 avec la distribution n°2

Analyse des résultats

L'observation des résultats obtenus sur ces deux arborescences permet de tirer plusieurs enseignements :

- **Répartition des clusters positifs :** Dans les deux distributions, les clusters positifs (en rouge) se situent préférentiellement en profondeur dans l'arborescence, confirmant le biais identifié lors du choix de la méthode de sélection des candidats. Cette tendance est cohérente avec notre hypothèse selon laquelle du contenu sensible est plus susceptible d'être dissimulé dans des branches profondes que proche de la racine.
- **Impact du nombre de clusters :** La première distribution (un seul cluster) génère une localisation compacte des nœuds positifs, concentrés autour d'une même zone de l'arborescence. En revanche, la seconde distribution (trois clusters) disperse les nœuds positifs sur plusieurs sous-arbres, créant une couverture plus large et plus représentative de la structure globale.

En résumé, la combinaison des méthodes de sélection et des distributions appliquées permet d'obtenir des environnements de test réalistes, où les clusters positifs reproduisent des scénarios plausibles de dissimulation en profondeur. La deuxième distribution, en introduisant plusieurs clusters, enrichit la diversité des cas étudiés et renforce la pertinence des environnements générés.

3 Hyperparamètres

De nombreux hyperparamètres sont échantillonnés aléatoirement entre une borne inférieure et une borne supérieure. Ces bornes ont été choisies de manière empirique, dans l'objectif de produire des arborescences réalistes. Toutefois, cela introduit un biais potentiel sur la forme des graphes et sur la distribution des fichiers positifs, pouvant provoquer une légère différence de performance entre une arborescence réelle et notre environnement de test.

Pour renforcer le réalisme des environnements de test, une collaboration avec les enquêteurs du CNAIP permettrait d'affiner ces intervalles de manière plus représentative.

3.1 Définitions des lois utilisées

Loi uniforme discrète. Soit $X \sim \mathcal{U}_d(a, b)$ une variable aléatoire suivant une loi uniforme discrète. On a :

- **Paramètres** : $a, b \in \mathbb{N}$ tels que $b \geq a$
- **Loi de probabilité** :

$$\forall k \in \llbracket a, b \rrbracket, \quad \mathbb{P}(X = k) = \frac{1}{b - a + 1}$$

- **Espérance** :

$$\mathbb{E}(X) = \frac{a + b}{2}$$

- **Variance** :

$$\mathbb{V}(X) = \frac{(b - a)(b - a + 2)}{12}$$

Loi uniforme continue. Soit $X \sim \mathcal{U}_c(a, b)$ une variable aléatoire suivant une loi uniforme continue. On a :

- **Paramètres** : $a, b \in \mathbb{R}$ tels que $b > a$
- **Densité de probabilité** :

$$f_X(x) = \begin{cases} \frac{1}{b - a} & \text{si } x \in [a, b] \\ 0 & \text{sinon} \end{cases}$$

- **Espérance** :

$$\mathbb{E}(X) = \frac{a + b}{2}$$

- **Variance** :

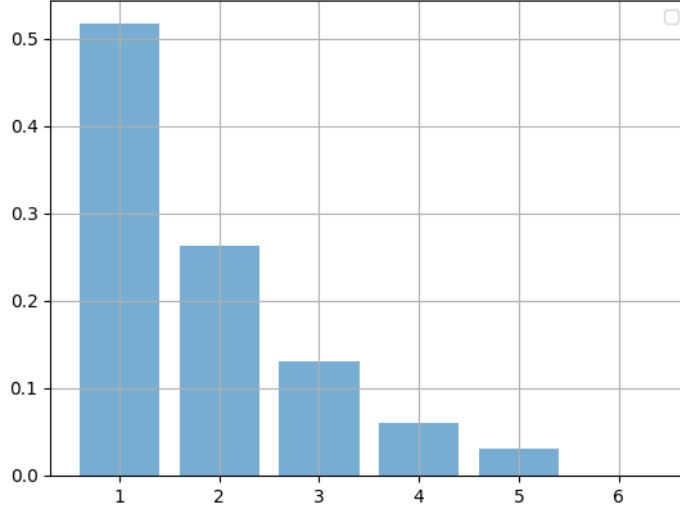
$$\mathbb{V}(X) = \frac{(b - a)^2}{12}$$

Loi géométrique tronquée. Soit $X \sim \mathcal{G}_t(n, r)$ une variable aléatoire suivant une loi géométrique discrète normalisée et tronquée à n termes. On a :

- **Paramètres** : $n \in \mathbb{N}^*, r \in [0, 1]$
- **Loi de probabilité** :

$$\mathbb{P}(X = k) = \begin{cases} \frac{(1 - r) r^{k-1}}{1 - r^n} & \text{si } k \in \llbracket 1, n \rrbracket \\ 0 & \text{sinon} \end{cases}$$

— Distribution de $X \sim \mathcal{G}_t(5, 0,49)$:



3.2 Hyperparamètres pour la génération des arborescences

Hyperparamètre	Loi	Description
n_{nodes}	$\mathcal{U}_d(50, 200)$	Contrôle le nombre de dossiers dans l'arborescence
$\text{max}_{\text{children}}$	$\mathcal{U}_d(2, 10)$	Contrôle le nombre de dossiers enfants maximum qu'un dossier peut avoir
$\text{max}_{\text{height}}$	$\mathcal{U}_d(5, 15)$	Contrôle la profondeur maximale d'une branche de l'arborescence

3.3 Hyperparamètres pour la distribution des fichiers

Ces paramètres sont appliqués indépendamment à chaque distribution sur les copies d'une même arborescence, permettant de tester différentes distribution de fichiers sur une structure donnée.

Hyperparamètre	Loi	Description
n_{clusters}	$\mathcal{G}_t(5, 0,49)$	Nombre de clusters de fichiers positifs
$n_{\text{pos_folder}}$	$\mathcal{U}_d(n_{\text{clusters}}, 15)$	Nombre de dossiers contenant des fichiers positifs
$n_{\text{pos_img}}$	$\mathcal{U}_d(2 \times n_{\text{pos_folder}}, 200)$	Nombre total d'images positives
$n_{\text{neg_img}}$	$\mathcal{U}_d(300, 900)$	Nombre total d'images négatives
max_{gap}	$\mathcal{U}_d(2, 6)$	Distance maximale pour la formation des clusters
$\text{mean}_{\text{exp_dist}}$	$\mathcal{U}_c(1, 15)$	Distance moyenne attendue entre les nœuds d'un cluster

Conclusion

Le système de génération d'environnements de test développé constitue un levier essentiel pour évaluer et optimiser l'agent de Reinforcement Learning dans le cadre du projet ODIP. En combinant la création d'arborescences réalistes avec une distribution contrôlée des fichiers, il devient possible de simuler des scénarios variés et représentatifs, offrant ainsi un terrain d'expérimentation robuste.

À moyen terme, plusieurs pistes d'amélioration sont envisageables : affiner le paramétrage grâce à l'expertise d'acteurs de terrain, introduire de nouveaux mécanismes de bruit pour renforcer le réalisme des environnements, et élargir la diversité des distributions étudiées. Ces évolutions renforceront encore la pertinence du système et en feront un outil solide pour accompagner le déploiement opérationnel d'ODIP.

ANNEXES

A Métriques d'évaluation de la stratégie d'ordonnancement

Cette annexe présente les métriques utilisées pour évaluer l'efficacité de notre stratégie d'ordonnancement intelligent des images. Ces métriques permettent de quantifier les performances de l'agent RL dans sa capacité à détecter rapidement les contenus à caractère pédopornographique.

Notations

Soit :

- $N \in \mathbb{N}$: nombre total d'images,
- $P \subset \{1, \dots, N\}$: ensemble des indices des images positives, avec $|P| = n_p$,
- Fonction d'étiquetage : $y : \{1, \dots, N\} \rightarrow \{0, 1\}$, avec $y(i) = 1$ si et seulement si $i \in P$,
- $\pi_i : \{1, \dots, N\} \rightarrow \{1, \dots, N\}$: permutation définie à l'étape i , représentant l'ordre dans lequel les images sont examinées à cette étape. On suppose que :

$$\forall i \in \{1, \dots, N\}, \forall j < i, \forall k \leq j, \quad \pi_i(k) = \pi_j(k)$$

autrement dit, les permutations successives π_i préservent l'ordre des choix passés.

Métrique 1 : Position relative du premier positif

Cette métrique mesure la rapidité avec laquelle le premier contenu illicite est détecté :

$$M1 = \frac{1}{N} \cdot \min \{i \in \{1, \dots, N\} \mid y(\pi_i(i)) = 1\}$$

où $M1 \in]0, 1]$. Plus $M1$ est proche de 0, plus la détection du premier positif est rapide.

Métrique 2 : Courbe de récupération cumulée des positifs

Pour chaque étape $i \in \{1, \dots, N\}$, on définit :

$$R(i) = \frac{1}{n_p} \sum_{j=1}^i y(\pi_j(j))$$

La fonction $R(i)$ représente la fraction des positifs trouvés après i itérations.

Métrique 3 : Aire relative sous la courbe de récupération (AUC normalisée)

L'aire brute sous la courbe R est définie comme :

$$AUC_{\text{brute}} = \sum_{i=1}^N R(i)$$

On définit l'aire maximale atteignable dans le cas idéal où tous les positifs sont détectés en premier :

$$AUC_{\text{max}} = n_p \cdot \left(N - \frac{n_p - 1}{2} \right)$$

La métrique normalisée est alors :

$$M3 = \frac{AUC_{\text{brute}}}{AUC_{\text{max}}} \in [0, 1]$$

Cette métrique est invariante au nombre de positifs et à la taille du dataset, ce qui permet une comparaison équitable entre différents environnements de test.

Métrique 4 : Effort relatif pour atteindre un pourcentage donné de positifs

Pour un seuil $\alpha \in]0, 1]$, on définit :

$$M4_{\alpha} = \min \left\{ \frac{i}{N} \mid R(i) \geq \alpha \right\}$$

Cette métrique indique le pourcentage d'images nécessaires pour découvrir au moins $\alpha \cdot 100\%$ des positifs.

Interprétation et utilisation

Ces métriques permettent une évaluation multidimensionnelle de la stratégie d'ordonnancement :

- **M1** privilégie la détection rapide du premier élément critique,
- **M3** évalue la performance globale sur l'ensemble de la recherche,
- **M4** permet d'analyser les performances à différents seuils de complétude.

L'utilisation conjointe de ces métriques offre une vision complète de l'efficacité de l'agent RL dans différents contextes opérationnels, permettant d'optimiser la stratégie selon les priorités spécifiques.

B Condition minimale sur le nombre initial de feuilles

Cette section établit une borne inférieure sur le nombre initial de feuilles nécessaires pour construire un arbre connexe comportant au moins un nombre total de nœuds donné, dans un scénario optimiste où chaque opération de fusion agrège le nombre maximal d'enfants possible.

Notations

Soit :

- $n_{initial} \in \mathbb{N}^*$: nombre initial de feuilles (arbres),
- $n \in \mathbb{N}$: nombre d'étapes de fusion,
- $u_n = n_{initial} + n(-max_{children} + 1)$: nombre de forêts restantes après n fusions,
- $v_n = n_{initial} + n$: nombre total de nœuds présents après n fusions.

Objectif

Déterminer la plus petite valeur de $n_{initial}$ telle qu'il existe un $n \in \mathbb{N}$ vérifiant simultanément :

$$(\text{connectivité}) \quad u_n = 1, \quad (\text{taille minimale}) \quad v_n \geq n_{nodes}$$

La première contrainte garantit que toutes les arbres ont été fusionnées en un arbre unique. La seconde impose que l'arbre final contient un nombre de nœuds suffisant.

Calcul de la borne minimale sur $n_{initial}$

Réolvons la contrainte de connectivité :

$$u_n = n_{initial} + n(-max_{children} + 1) = 1 \quad \Rightarrow \quad n_{initial} = 1 + n(max_{children} - 1)$$

Injectons dans l'expression de v_n :

$$v_n = n_{initial} + n = 1 + n(max_{children} - 1) + n = 1 + n \cdot max_{children}$$

La contrainte $v_n \geq n_{nodes}$ devient :

$$1 + n \cdot max_{children} \geq n_{nodes} \quad \Rightarrow \quad n \geq \left\lceil \frac{n_{nodes} - 1}{max_{children}} \right\rceil$$

On en déduit la condition minimale sur $n_{initial}$:

$$n_{initial} \geq 1 + \left\lceil \frac{n_{nodes} - 1}{max_{children}} \right\rceil (max_{children} - 1)$$